

Assignment16(CSB24084)

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;

        printf("Producer produced item %d at position %d\n", i, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg)
{

```

```

int item;

for (int i = 1; i <= ITEMS; i++)
{
    sem_wait(&full);

    pthread_mutex_lock(&mutex);

    item = buffer[out];

    printf("Consumer consumed item %d from position %d\n", item, out);

    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);

    sem_post(&empty);

    sleep(2);
}

return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    pthread_mutex_destroy(&mutex);

```

```
    return 0;  
}
```

Synchronization prevents inconsistent behavior by ensuring that only one thread accesses the shared resource at a time. In the producer-consumer system, semaphores and mutexes coordinate the execution of threads so that the producer waits when the buffer is full and the consumer waits when the buffer is empty. Functions like `sem_wait()` and `sem_post()` control thread signaling, while the mutex protects the shared buffer from simultaneous access. This prevents race conditions, data corruption, and incorrect execution, ensuring safe and consistent communication between threads.